

get when it becomes available. Now it's more of a first-class *program* that we can *run*. So let's rename our *get* function to *run*, and dictate that this is where the parallelism actually gets implemented:

```
def run[A](a: Par[A]): A
```

Because *Par* is now just a pure data structure, *run* has to have some means of implementing the parallelism, whether it spawns new threads, delegates tasks to a thread pool, or uses some other mechanism.

7.2 Picking a representation

Just by exploring this simple example and thinking through the consequences of different choices, we've sketched out the following API.

Listing 7.4 Basic sketch for an API for *Par*

Marks a computation for concurrent evaluation by *run*.

```
def unit[A](a: A): Par[A]
def map2[A,B,C](a: Par[A], b: Par[B])(f: (A,B) => C): Par[C]
def fork[A](a: => Par[A]): Par[A]
def lazyUnit[A](a: => A): Par[A] = fork(unit(a))
def run[A](a: Par[A]): A
```

Creates a computation that immediately results in the value *a*.

Combines the results of two parallel computations with a binary function.

Wraps the expression *a* for concurrent evaluation by *run*.

Fully evaluates a given *Par*, spawning parallel computations as requested by *fork* and extracting the resulting value.

We've also loosely assigned meaning to these various functions:

- *unit* promotes a constant value to a parallel computation.
- *map2* combines the results of two parallel computations with a binary function.
- *fork* marks a computation for concurrent evaluation. The evaluation won't actually occur until forced by *run*.
- *lazyUnit* wraps its unevaluated argument in a *Par* and marks it for concurrent evaluation.
- *run* extracts a value from a *Par* by actually performing the computation.

At any point while sketching out an API, you can start thinking about possible *representations* for the abstract types that appear.



EXERCISE 7.2

Before continuing, try to come up with representations for *Par* that make it possible to implement the functions of our API.